Projet AWD - JobBoard

Document de contexte (validation intermediaire, seance 4)

Ce document resume tout le travail deja fait afin de reprendre le projet dans une autre conversation. Etudiant : **Adam** (travail **individuel**). Depot GitHub : **https://github.com/adam20031803/AWD**

1. Consigne du projet (resume)

• **Git** : depot avec dossiers Backend (microservices), Frontend, Documentation (presentation, diagramme d'architecture globale, principaux diagrammes, dossier technique Word ou LaTeX) et README.md. L'enseignant doit etre ajoute comme collaborateur.

• **Presentation** (10 a 15 min + 10 min de discussion) : idee et problematique, acteurs et cas d'utilisation, fonctionnalites, architecture globale (API Gateway, securite, donnees, communications), microservices et choix de bases de donnees, technologies, repartition des taches, organisation Git et outil de gestion, planning et risques.

• **Partie individuelle** (1 min par microservice) : diagramme de classes, exemples de communications synchrones et asynchrones, contrats d'API (endpoints, codes de reponse, evenements). Le microservice commun est developpe par toute l'equipe.

• **Blackboard** (un seul membre) : lien du depot Git, document technique, presentation, diagramme d'architecture globale.

• Le projet n'a pas besoin d'etre termine : une conception coherente, un depot clair et un plan realiste suffisent.

• Toute absence non justifiee ou non-implication = zero. Chaque membre doit pouvoir expliquer toutes les parties.

2. Situation

• Pas d'idee imposee, pas de groupe : l'etudiant travaille seul. Il faut **confirmer par email avec l'enseignant** que le travail individuel est accepte et quel perimetre est attendu.

• Projet choisi : **Job Board** (plateforme de recherche d'emploi), reduit a 3 microservices metier + API Gateway + Eureka.

3. Sujet, acteurs et fonctionnalites

Les offres d'emploi et candidatures sont dispersees sur plusieurs canaux. JobBoard centralise la publication d'offres par les recruteurs et le depot de candidatures par les candidats.

Acteur	Fonctionnalites
Candidat	Inscription, connexion, recherche d'offres, candidature, suivi du statut
Recruteur	Publier / modifier / supprimer des offres, consulter et traiter les candidatures
Administrateur	Gerer les utilisateurs, moderer les offres

4. Architecture et microservices

Service	Responsabilite	Base de donnees	Justification
api-gateway	Point d'entree unique, validation JWT, routage	-	Securite centralisee
discovery-server	Registre Eureka	-	Decouverte dynamique
auth-service (commun)	Comptes, roles (CANDIDATE, RECRUITER, ADMIN), emission des JWT	PostgreSQL	Donnees relationnelles, integrite
offer-service	Offres d'emploi	MongoDB	Schema flexible, recherche textuelle
application-service	Candidatures, statuts, publication d'evenements	PostgreSQL	Transactions, unicite (un candidat ne postule qu'une fois)

Principe : une base par service (database per service). Pas de jointure entre services : references par identifiant (recruiterId, offerId, candidateId).

Securite : le client n'appelle que l'API Gateway, qui valide le JWT signe par auth-service et transmet identite et role aux services. Mots de passe haches avec BCrypt.

Communications :

• **Synchrone (REST)** : application-service appelle offer-service (GET /api/offers/{id}) pour verifier que l'offre existe avant d'enregistrer une candidature.

• **Asynchrone (RabbitMQ)** : application-service publie application.submitted et application.status.changed. Un notification-service optionnel pourra les consommer.

Schema d'architecture (texte)

```
Client Angular --HTTPS+JWT--> API Gateway
   API Gateway --> auth-service        (PostgreSQL auth_db)
   API Gateway --> offer-service       (MongoDB offers_db)
   API Gateway --> application-service (PostgreSQL applications_db)
   Eureka : tous les services s'y enregistrent
   application-service --REST synchrone--> offer-service
   application-service --evenement--> RabbitMQ --> notification-service (optionnel)
```

5. Technologies

Couche	Choix
Backend	Spring Boot 3, Spring Cloud Gateway, Eureka
Securite	Spring Security + JWT
Bases de donnees	PostgreSQL (auth, applications), MongoDB (offres)
Messagerie	RabbitMQ
Frontend	Angular
Conteneurs	Docker / Docker Compose
Gestion de projet	GitHub Projects (Kanban) + Issues

Hypothese : stack Spring Boot + Angular. A adapter si l'etudiant utilise autre chose (Node.js, Django, React...).

6. Diagrammes de classes

Service	Classe	Attributs
auth-service	User	id, email, passwordHash, fullName, role (CANDIDATE | RECRUITER | ADMIN), createdAt
offer-service	Offer	id, title, description, location, type (CDI | CDD | INTERNSHIP | FREELANCE), salary, recruiterId, publishedAt, active
application-service	Application	id, offerId, candidateId, coverLetter, cvUrl, status (PENDING | REVIEWED | ACCEPTED | REJECTED), appliedAt

7. Contrats d'API

auth-service

Methode	Endpoint	Description	Codes
POST	/api/auth/register	Creer un compte	201, 400, 409
POST	/api/auth/login	Connexion, retourne un JWT	200, 401
GET	/api/users/me	Profil connecte	200, 401
GET	/api/users/{id}	Infos d'un utilisateur	200, 401, 404
DELETE	/api/users/{id}	Supprimer un compte (ADMIN)	204, 403, 404

offer-service

Methode	Endpoint	Role	Codes
GET	/api/offers?keyword=&location;=&type;=	Tous	200
GET	/api/offers/{id}	Tous	200, 404
POST	/api/offers	RECRUITER	201, 400, 401, 403
PUT	/api/offers/{id}	RECRUITER (proprietaire)	200, 403, 404
DELETE	/api/offers/{id}	RECRUITER / ADMIN	204, 403, 404

application-service

Methode	Endpoint	Role	Codes
POST	/api/applications	CANDIDATE	201, 400, 404 (offre), 409 (deja postule)
GET	/api/applications/me	CANDIDATE	200
GET	/api/applications/offer/{offerId}	RECRUITER	200, 403
PATCH	/api/applications/{id}/status	RECRUITER	200, 403, 404

Evenements RabbitMQ : application.submitted {applicationId, offerId, candidateId, recruiterId, date} ; application.status.changed {applicationId, candidateId, newStatus}.

Sequence : candidature

```
1. Candidat -> Gateway : POST /api/applications (JWT)
2. Gateway valide le JWT, transmet a application-service
3. application-service -> offer-service : GET /api/offers/{id}   (synchrone)
4. offer-service repond 200 (offre) ou 404
5. application-service enregistre la candidature (statut PENDING)
6. application-service publie application.submitted dans RabbitMQ (asynchrone)
7. application-service repond 201 Created au candidat
```

8. Planning previsionnel et risques

Semaine	Objectif
4	Conception, depot Git, presentation
5	Discovery server, Gateway, auth-service
6	offer-service
7	application-service + RabbitMQ
8	Frontend Angular
9	Docker Compose, tests, finalisation

Risque	Mitigation
Charge de travail (projet seul)	Perimetre reduit a 3 services, notification-service optionnel
Complexite de la securite JWT	Validation uniquement au Gateway au debut
Integration RabbitMQ	Prototype tot (semaine 7)
Retard sur le frontend	Interface minimale, priorite a l'API

9. Organisation Git

```
AWD/
|-- Backend/
|   |-- api-gateway/
|   |-- discovery-server/
|   |-- auth-service/          (microservice commun)
|   |-- offer-service/
|   `-- application-service/
|-- Frontend/
|-- Documentation/
|   |-- Presentation/          (plan-presentation.md)
|   |-- Diagrams/              (architecture, use-cases, classes, sequences - Mermaid)
|   `-- dossier-technique.md
`-- README.md
```

• Branches : main (stable), develop (integration), feature/<service>-<fonction>.

• Commits : feat(offer): add search endpoint.

• Outil de gestion : GitHub Projects (Kanban : To do / In progress / Done).

10. Etat d'avancement

Element	Etat
Depot GitHub cree (public, branche main)	Fait
Squelette du depot, README, diagrammes, dossier technique en Markdown (fichier AWD-skeleton.zip)	Fait
Copie des fichiers dans le clone C:\Users\adamb\AWD, commit et push vers main	**En cours** : un premier commit a ete fait par erreur dans C:\Users\adamb (depot parasite) ; il faut cloner/copier dans C:\Users\adamb\AWD (xcopy), puis git add, commit, push origin main
Ajouter l'enseignant comme collaborateur (Settings > Collaborators)	A faire
Email a l'enseignant pour valider le travail individuel	A faire
Dossier technique en Word (.docx) ou LaTeX	A faire
Presentation PowerPoint (10-15 min)	A faire
Diagramme d'architecture globale en image / PDF	A faire
Rendu Blackboard (lien Git, dossier technique, presentation, diagramme)	A faire
Code des microservices	Apres la validation

11. Notes techniques utiles

• Notebook Fashion MNIST (autre cours) : les cellules ont ete completees (Sequential 256-128-10, loss sparse_categorical_crossentropy, optimizer sgd, 10 epochs). TensorFlow ne s'installait pas : le noyau Jupyter etait en Python 3.14 ; solution : environnement Python 3.11 ou 3.12, ou Google Colab. L'installation (%pip install) doit etre dans une cellule separee, avant les imports.

• Environnement de l'etudiant : Windows, VS Code, Git Bash et Invite de commandes.

• L'etudiant prefere des reponses en **francais**.